

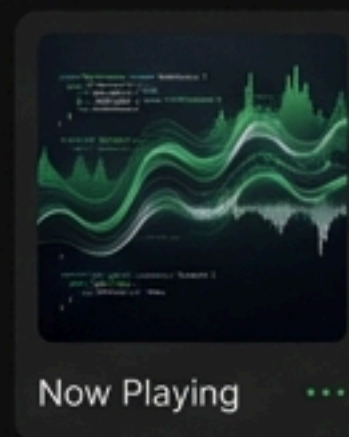
Programación Orientada a Objetos: La Edición Spotify

```
class MusicStream extends AudioSource {  
  play(Track song) {  
    this.bufferSize = 4096;  
    this.bufferSize = this.TrackStreamweng();  
    this.bufferSize;;  
  }  
}
```

```
interface Playlist {  
  public Iterator<Track> iterator();  
  public void add(Track track);  
}
```

```
interface Playlist implements Iterable {  
  play(Track song) {  
    this.bufferSize;  
  }  
}
```

NIVEL EXPERTO



Deja de escribir código. Empieza a diseñar sistemas.



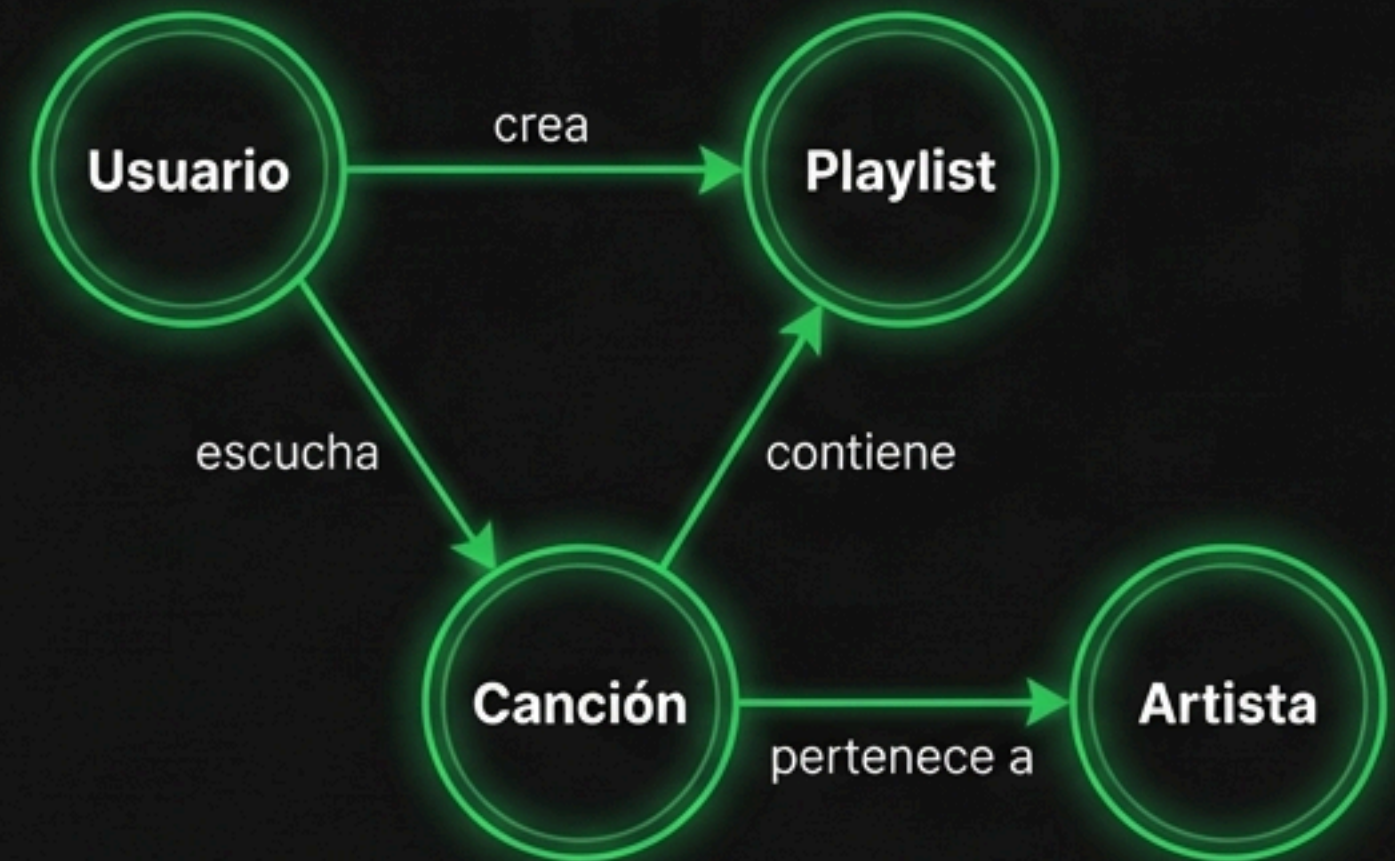
El cambio de mentalidad: De Scripter a Arquitecto

JUNIOR

```
function play_song() { ... }  
function calculate_time() { ... }  
function get_artist() { ... }  
  
// Funciones sueltas sin estructura
```

Piensa en líneas de script.

SENIOR (Arquitecto)



Piensa en entidades interconectadas.

Spotify no piensa en funciones. Spotify piensa: '¿Qué entidades existen en el universo de la música?'

El Dominio: El Mapa del Territorio

Antes de escribir código, identificamos las entidades clave.



La Clase es el Contrato

La promesa de lo que una canción DEBE tener.

class Track

PROPIEDADES (DATOS)

```
title: String  
artist: String  
duration: Integer  
genre: String  
audio_features: Object { tempo, energy }
```

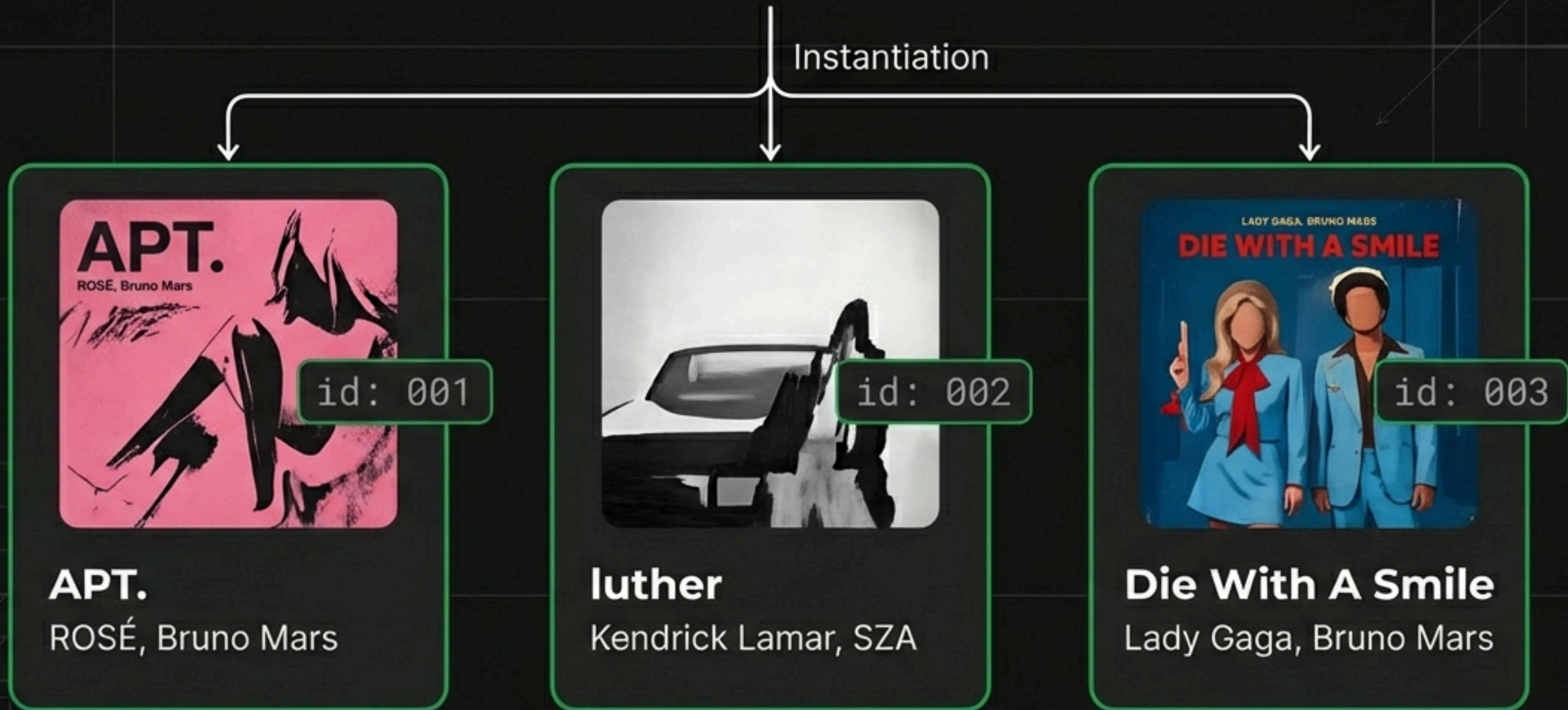
ESTADO INTERNO

```
is_playing: Boolean  
popularity_score: Float
```

El contrato define la estructura. Si un objeto no cumple esto, no es una canción.

Instancias: El Contrato en Acción

Mismo contrato (Clase), distintos datos (Valores).



Estado: El Objeto está Vivo

El estado vive DENTRO del objeto. No está disperso en el código global.



T = 0s (Inicio)

T = 30s (Reproduciendo)

User Action: Click Heart



Métodos vs. Funciones: Intención sobre Ejecución

✗ JUNIOR (Procedural)

`play_song(song)`

Una función externa actúa sobre la data. La canción es pasiva.

✓ SENIOR (Orientado a Objetos)

`song.play()`

La canción es dueña de su comportamiento. Ella sabe cómo reproducirse.

Definimos comportamientos que pertenecen a la entidad.

Anatomía de un Método: track.play()

```
def play(self):
```

Valida estado (¿Ya suena?)

Cambia estado: `is_playing = True`

Log Analytics (Historial)

Notificar al Player

Disparar Métricas (Popularidad)

```
return True
```

Un método encapsula lógica compleja. El resto del sistema solo llama a `.play()` y confía.

Encapsulación: El 'Bouncer' de tus Datos



(A) Peligroso / Rompe Consistencia

`track.is_playing = True`



(B) Valida Reglas / Seguro

`track.play()`



Method



"No toques mis datos, usa mis métodos."

La Playlist: Más que una Lista

Un contenedor sofisticado con reglas de negocio.

owner: User

total_duration: TimeDelta

followers: int

is_collaborative: bool



tracks: List<Track>

Method: shuffle()

Method: add_track(track)

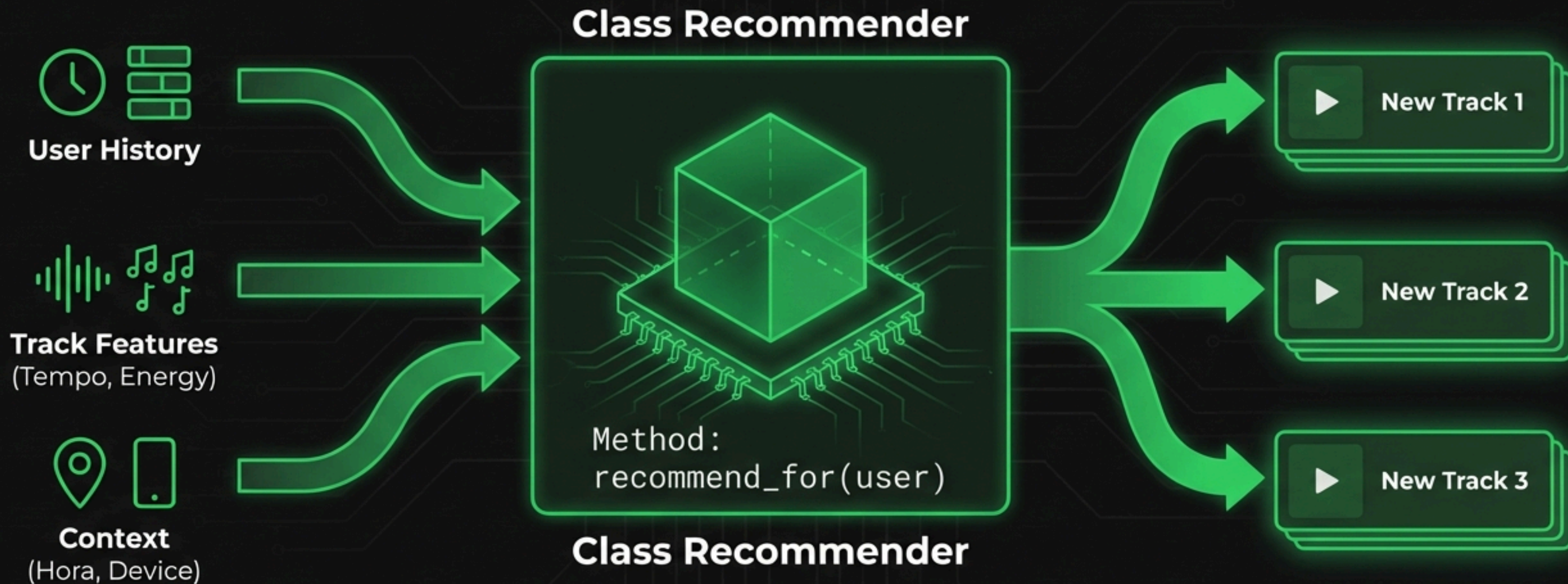
Method: share()

El Ecosistema: Interacción de Objetos

Un clic dispara una conversación entre objetos.

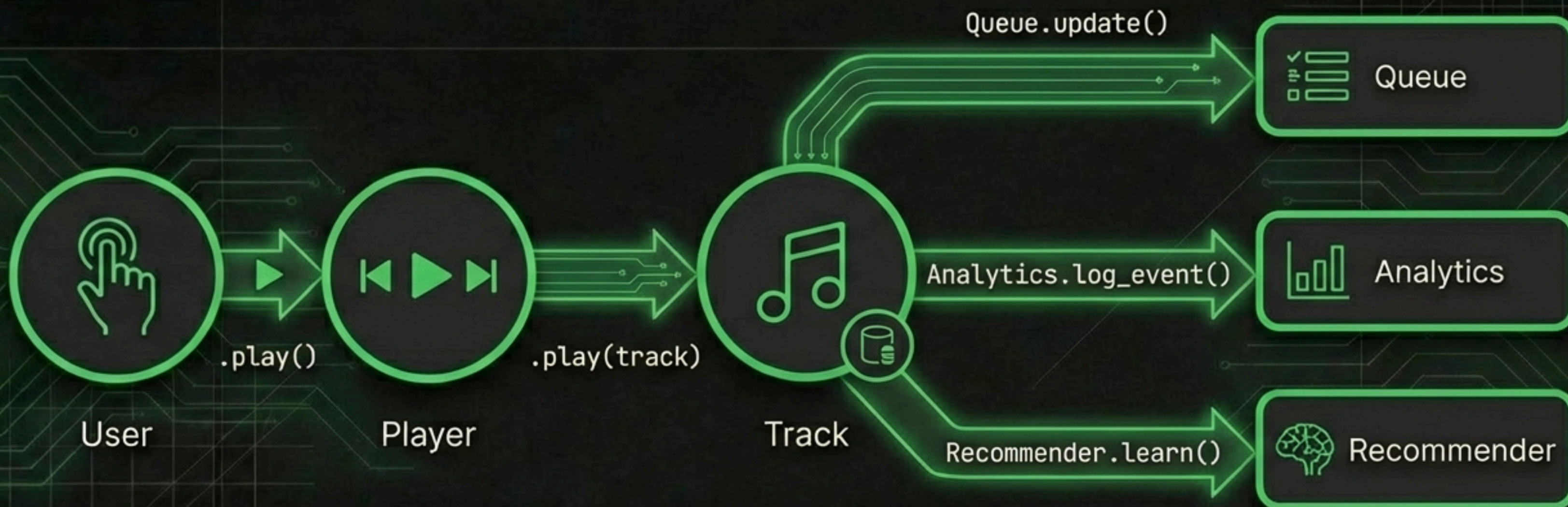


El Recomendador: OOP + Inteligencia Artificial



OOP organiza la data; el Machine Learning toma las decisiones.

La Sinfonía Completa: Flujo End-to-End



Todo es interacción. No hay scripts sueltos. Es un sistema vivo.

Tu Nueva Mentalidad

JUNIOR	EXPERTO
✓ Piensa en código ✓ Piensa en funciones ✓ Ve una lista de canciones ➤ Meta: Que funcione	➤ Piensa en dominio ➤ Piensa en contratos ➤ Ve responsabilidades y colaboraciones ➤ Meta: Que escale

Esta es la lista de verificación para tu próxima promoción.

Spotify no es una app de canciones.

**Es un sistema de objetos que modela
el comportamiento humano.**